Automated API to Snowflake Data Pipeline using AWS S3, Snowpipe & Airflow

Project Title

Cloud-Native Event-Driven API Data Pipeline to Snowflake Using AWS & Airflow

Abstract

This project presents the design and implementation of a cloud-native,event-driven data ingestion pipeline that extracts data from an external API, stores it in AWS S3, and automatically loads it into Snowflake using Snowpipe. The pipeline is orchestrated using Apache Airflow to ensure reliability, automation, and scalability.

The system leverages AWS IAM for secure integration between S3 and Snowflake, enabling seamless and secure data movement without hardcoded credentials. Incremental loading and UPSERT logic ensure data consistency and eliminate duplication.

The architecture demonstrates a production-ready, scalable, and secure modern data engineering workflow suitable for enterprise-grade applications.

Executive summary

This repository implements a focused, end-to-end ingestion pipeline for a concrete demo workload: fetch 500 user records from the Random User Generator API (commonly `https://randomuser.me/api/?results=500`), serialize them to JSON (recommended) and/or CSV, upload the files to Amazon S3, ingest them into Snowflake via Snowpipe (either auto-ingest using S3 event notifications → SQS, or Snowpipe REST), land raw rows in a staging table, and then upsert into a target table using a deterministic `MERGE` keyed by `login.uuid`.

1. Introduction

In modern data-driven organizations, real-time and automated data ingestion is critical for analytics, reporting, and business intelligence. Companies often consume data from external APIs, but transforming and loading that data into a centralized data warehouse requires scalable and secure architecture.

Traditional manual data ingestion methods are:

- Error-prone
- Not scalable
- Difficult to monitor
- Lacking automation

This project implements a fully automated pipeline using:

- Python for API extraction
- AWS S3 for data storage
- Snowpipe for automatic ingestion
- Snowflake as the data warehouse
- Apache Airflow for orchestration
- Snowflake : SQL examples use generally available features (stages, pipes, COPY, MERGE).
- File format is JSON (NDJSON) and CSV options and associated FILE FORMAT objects.
- RandomUser parameters are assumed per the official docs: results supports up to 5,000 per request; deterministic pagination requires consistent seed + page (1-based).

- If you choose **Snowpipe auto-ingest,** you will configure **S3 event notifications** (typically `s3:ObjectCreated:*`) to the **Snowflake-managed SQS queue** assigned to your pipe; note S3 notifications are **at-least-once** and can be duplicated/out of order.

2. Problem Statement

Organizations frequently rely on external APIs as primary data sources. However, several challenges arise:

1. Lack of automation in data extraction
2. Manual loading of data into warehouses
3. Poor scalability
4. Data duplication during ingestion
5. Security risks due to exposed credentials
6. No centralized monitoring system

The objective is to design a secure, scalable, and automated pipeline that:

- Extracts data from APIs
- Stores raw data reliably
- Automatically loads into Snowflake
- Maintains data integrity
- Ensures secure cross-cloud communication

3. Objectives

The key objectives of this project are:

- Design an end-to-end automated data ingestion pipeline
- Implement event-driven data loading using Snowpipe
- Use AWS IAM for secure integration
- Enable incremental data loading using UPSERT logic
- Orchestrate workflows using Apache Airflow
- Build a scalable and production-ready cloud architecture

4. System Architecture

Architecture Overview

The architecture follows an event-driven ELT design:

API → Python Script → AWS S3 → Snowpipe → Snowflake Stage → Target Tables

Key Layers:

1. Data Source Layer (API)
2. Landing Layer (AWS S3)
3. Ingestion Layer (Snowpipe)
4. Processing Layer (Snowflake)
5. Orchestration Layer (Airflow)

Concise end-to-end flow

- Python calls the API and extracts the results[] array (each element is a nested JSON user object containing login.uuid, name, location, etc.).
- Python writes 500 user objects to JSON (.json or .ndjson)
- Python uploads the file to S3 under a unique object key (example: randomusers/run_date=YYYY-MM-DD/run_id=<uuid>/users.json.).
- Snowpipe auto-ingest: S3 sends ObjectCreated notifications to the Snowflake SQS queue; Snowpipe queues the file and loads it to the staging table using the COPY INTO statement defined in the pipe, using Snowflake-provided compute.
- A transform step runs MERGE from staging to target; to stay deterministic, the source is deduplicated so one staging row maps to one target key.
- Snowpipe auto-ingest for S3 is built around SQS notifications for an S3 bucket/prefix, and Snowflake's documented process flow is "stage files → S3 event notification → Snowpipe queue → Snowflake-provided warehouse loads into table.

5. Detailed Workflow Explanation

5.1 Data Extraction Layer

A Python script is developed to:

- Call the external API
- Retrieve data in JSON format
- Validate and transform the data
- Save the output as structured files (CSV/JSON)

Why Python?

- Flexible
- Lightweight
- Easily integrated with Airflow

5.2 Data Storage Layer (AWS S3)

Extracted files are stored in an AWS S3 bucket.

S3 serves as:

- Raw data landing zone
- Durable object storage
- Integration point with Snowflake

Advantages:

- High availability
- Cost-efficient
- Scalable storage
- Secure access control

5.3 Secure Integration (AWS IAM + Snowflake)

Snowflake uses an IAM role to access S3 securely.

Security Features:

- Role-based authentication
- No access keys stored in code
- Trust relationship between AWS and Snowflake
- Principle of least privilege

5.4 Snowpipe Auto-Ingestion

Snowpipe continuously monitors the S3 bucket.

When new files are detected:

- Snowpipe automatically triggers
- Files are copied into Snowflake staging tables

This is event-driven ingestion.

Benefits:

- Near real-time loading
- No manual intervention
- Scalable ingestion

5.5 Data Transformation & UPSERT Logic

After staging:

- MERGE statements are executed
- Data is incrementally updated
- Duplicate records are avoided

UPSERT ensures:

- Idempotent loading
- Data consistency
- Efficient incremental processing

5.6 Orchestration with Apache Airflow

Apache Airflow manages:

- API extraction scheduling
- Task dependencies
- Error handling
- Retry mechanisms
- Logging and monitoring

Airflow DAG ensures:

- Reliable execution
- Observability
- Fault tolerance

Python extractor and S3 uploader

RandomUser extraction notes that affect code & data quality

- You can request up to 5,000 users per request using the `results` parameter; this project uses `results=500`.
- The API supports repeatable datasets via `seed`; and pagination via `page` (1-based) combined with the same `seed` to reproduce the "same" paged results.
- The API can return a simple JSON error object when the service is offline or has server issues; robust code should detect that.
- The response schema is nested (example shows `login.uuid`, nested `location` objects, timestamps like `dob.date`); NDJSON preserves this structure for Snowflake `VARIANT` ingestion.

Python: fetch 500, write JSON and CSV, upload to S3 (with error handling)

it sets timeouts, retries transient errors, checks for the API-level error payload, and writes either JSON (recommended) or a small flattened CSV (useful for simple staging).

Why JSON is the "default" recommendation here:

- The API response is nested and semi-structured; JSON preserves nested fields without aggressive flattening.
- Snowflake's semi-structured ingestion behavior includes rules around nulls and mixed data types that can affect how elements are extracted into columnar storage; keeping raw VARIANT in staging avoids prematurely losing nuance and lets you normalize downstream.

Snowflake and Snowpipe setup

Snowpipe options for this project

Option A: Snowpipe auto-ingest (recommended steady-state)

- Snowpipe for S3 is triggered automatically using Amazon SQS notifications from the S3 bucket/prefix.
- Snowflake's documented flow is: stage files → S3 event notification informs Snowpipe via SQS → Snowflake-provided compute loads from queue into the table defined by the pipe.
- You may need SNS fanout (Snowflake calls it "Option 2") if an event notification already exists for the same bucket/prefix because AWS prohibits conflicting notifications for the same path; SNS acts as a broadcaster to multiple subscribers including Snowflake's queue.

Option B: Snowpipe REST (useful for backfills or when you can't configure S3 notifications)

- You call `insertFiles` with a list of staged file paths; success means Snowflake recorded the files for ingestion, not necessarily that ingestion already completed.
- REST calls use key-based authentication (Snowpipe ingestion service does not maintain client sessions), and Snowflake provides Java/Python SDKs for convenience.
- `insertFiles` supports up to 5000 files per request and can return 429 on rate limiting.

Secure S3 access: storage integration and least privilege

Snowflake strongly recommends using a storage integration (instead of embedding cloud keys) to access S3; integrations are first-class Snowflake objects that avoid passing secret keys and store an AWS IAM identity, and Snowflake associates the integration to an AWS IAM user created for your account.

Required S3 permissions (minimum read path)

- Snowflake documents the required S3 permissions needed to access files in a bucket/prefix: `s3:GetBucketLocation`, `s3:GetObject`, `s3:GetObjectVersion`, and `s3:ListBucket`.

Least-privilege principle

- AWS's security best practices explicitly recommend granting only the permissions required to perform the task (least privilege) and tightening policies over time.

Sample IAM policy and trust policy for Snowflake S3 access

The policy is a read-only loader scoped to a single bucket + prefix; it mirrors the read-only pattern Snowflake publishes as an "Alternative policy: Load from a read-only S3 bucket.

Snowflake SQL: integration, formats, stage, tables, pipe, and merge

Snowpipe auto-ingest assumes the target table already exists for loading. In this project, the pipe's target is the staging table.

CREATE STAGE is the Snowflake mechanism to define an external stage referencing S3, and if the path ends with /, objects under that "folder/prefix" are in scope.

Snowflake's S3 auto-ingest documentation shows CREATE PIPE ... AUTO_INGEST = TRUE AS COPY INTO ... FROM @stage FILE_FORMAT=... and explains AUTO_INGEST = TRUE means the pipe reads S3 event notifications sent to SQS when new data is ready.

Staging, loading, and upserts

COPY INTO best practices for this specific demo

Even though 500 records produce small files, Snowflake's guidance matters if you scale beyond a run:

- For performance and cost-efficient loading (bulk and Snowpipe), Snowflake recommends aiming for ~100–250 MB compressed files; very large files (e.g., 100 GB) are not recommended and error strategy (ON_ERROR) becomes consequential.
- For Snowpipe specifically, Snowflake notes there is queue-management overhead that increases with file count; following the file sizing recommendations reduces overhead relative to total loaded data.

For validation and error inspection:

- VALIDATION_MODE can validate files without loading, and Snowflake explicitly states it does not support COPY statements that transform data during load.
- For Snowpipe loads, VALIDATE_PIPE_LOAD validates files processed by Snowpipe in a time range and returns error details; activity is limited to the last 14 days.

Why staging exists and when to truncate it

In this project, staging is not “optional ceremony”; it is the system boundary between file ingestion (Snowpipe/COPY semantics) and relational correctness (dedupe + merge).

- Staging preserves the nested payload and load lineage, which is useful because the API output is nested and can contain fields you may want later without re-extracting.
- It allows deterministic merges by deduping per `login.uuid` before applying updates.

Truncation guidance :

After data is successfully loaded to the **Target Table**, we clean staging. Because staging should contain **only the latest batch of raw data**.

Otherwise:

- Duplicate merges
- Storage waste
- Data confusion
  - For a demo repo, it’s reasonable to truncate staging after a successful merge to keep storage small.
  - For a continuously running pipeline, prefer retention-by-time (delete partitions by `start_scan_time` / `loaded_at`) or use streams/tasks to consume changes incrementally, rather than unconditional truncation.

What is UPSERT?

UPSERT = UPDATE + INSERT. It is a database operation that:

- Updates existing records
- Inserts new records in a **single operation**.

Instead of checking manually whether a row already exists, the database **automatically decides**:

- If record **exists → UPDATE**
- If record **does not exist → INSERT**
- In **Snowflake**, UPSERT is implemented using the **MERGE statement**.

UPSERT (Update + Insert) is used to synchronize new incoming data with the existing target table. If a record already exists based on the primary key (user_id), the record is updated. If the record does not exist, a new record is inserted. This ensures the target table always contains the most up-to-date and duplicate-free dataset.

Why We Cannot Directly Load to Target Table?

Because raw API data can have problems like:

- **Null Values**
- **Duplicate Records**
- **Schema Mismatch**
- **Data Cleaning Needed**

What is Orchestration in This Project?

Orchestration means **automatically controlling and scheduling all steps of the data pipeline**.

Instead of running scripts manually, **Airflow runs them automatically in sequence**.

In this pipeline the order is:

Extract Data → Upload to S3 → Wait for Snowpipe → Merge into Target Table

Airflow ensures each step runs only after the previous step finishes successfully.

How Airflow Controls the Entire Pipeline

Airflow uses something called a **DAG (Directed Acyclic Graph)**.

A DAG is simply a **workflow definition** that tells Airflow:

- what tasks exist
- in which order they run
- when they should run

Example schedule : Run every day at 2 AM .Airflow will automatically start the pipeline.

Monitoring in Airflow

Airflow provides a **Web UI dashboard** where you can monitor pipeline status.

You can see:

- running pipelines
- failed tasks
- successful runs
- execution logs
-

Example task status:

Extract API ✓ Success

Upload to S3 ✓ Success

Snowpipe Load ✓ Success

Merge Table ✓ Success

If something fails, Airflow shows the error logs.

Why Airflow is Important in This Project?

Without Airflow you would need to manually run:

- Python extraction script
- S3 upload
- Snowflake merge query

every day.

Airflow automates everything.

Benefits:

- Automatic scheduling
- Task dependency management
- Failure retries
- Monitoring dashboard
- Logging and debugging

Apache Airflow is used to orchestrate the entire data pipeline by scheduling and managing tasks such as API extraction, uploading files to AWS S3, triggering Snowpipe ingestion, and executing the merge operation into the Snowflake target table. This automation ensures the pipeline runs reliably without manual intervention while providing monitoring and failure recovery capabilities.

6. Security Architecture

Security is implemented at multiple levels:

- IAM role-based S3 access
- Snowflake RBAC
- Encrypted S3 storage
- No hardcoded credentials
- Controlled stage access

This makes the system enterprise-ready.

7. Major Components Explanation

Component	Role
Python Script	Extract API data
AWS S3	Store raw data
IAM Role	Secure cross-service access
Snowpipe	Auto-ingestion mechanism

Snowflake Stage	Temporary landing area
Target Tables	Final analytical storage
Airflow DAG	Workflow automation

8. Advantages of This Architecture

- Fully automated
- Event-driven ingestion
- Secure IAM-based integration
- Scalable cloud-native design
- Incremental processing
- Production-ready architecture

9. Real-World Applications

- Financial transaction ingestion
- Marketing campaign data
- SaaS product analytics
- IoT data pipelines
- Real-time reporting systems

10. Future Enhancements

- Add data validation layer (Great Expectations)
- Integrate dbt for transformations
- Add CI/CD deployment
- Implement monitoring alerts
- Add data quality dashboards
- Introduce Data Lake layer

Conclusion

This project demonstrates the implementation of a secure, scalable, and automated cloud-native data pipeline using AWS, Snowflake, and Apache Airflow. The architecture follows modern ELT principles and event-driven ingestion, making it suitable for enterprise-grade production environments.

It effectively solves common challenges related to manual ingestion, data duplication, security, and scalability.